TrafficRule

A Domain-Specific Language and Compiler for Traffic Rule Enforcement

Course Code	BCSE307L
Course Name	Compiler Design Laboratory
Name	Alan James
Registration No.	24BCT0181

Fall Semester 2026–27

1 Abstract

TrafficRule is a small domain-specific language for writing and checking traffic rules. The main idea is to allow conditions such as speed limits, traffic signal states and vehicle positions to be written in a simple programming language.

A TrafficRule program is processed through different compiler stages. These include lexical analysis, syntax analysis, Abstract Syntax Tree construction, semantic analysis, symbol table management, intermediate code generation and basic optimization. The final rules can then be evaluated using simulated traffic data.

For example, a rule can be written as:

```
IF speed > 60 {
    ISSUE_FINE
}
```

The project does not aim to build an actual traffic enforcement system. There will be no dependency on traffic cameras, GPS or real-time traffic databases. Instead, sample traffic values will be used so that the focus remains on the compiler concepts involved.

2 Introduction

A compiler converts a program written by a user into a form that can be processed by a machine. This is done through several stages such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation and optimization.

In the Compiler Design course, these stages are usually studied separately. TrafficRule is an attempt to put these concepts together in one small project.

The language is designed around traffic rules. A user can define values such as vehicle speed, signal state and position and then write conditions based on them.

For example:

```
speed = 75

IF speed > 60 {
    ISSUE_FINE
}
```

The lexer will identify the keywords, identifiers, numbers and operators. The parser will then check whether the statements follow the grammar of the language. After that, an AST can be created and semantic checks can be performed.

The valid program can then be converted into an intermediate form and executed using simulated traffic data.

This makes the project useful for demonstrating how the different parts of a compiler work together.

3 Problem Statement

Traffic rules are normally written as conditions and actions. For example, a vehicle travelling above a certain speed can be considered a violation. These rules can be represented using a small programming language.

The problem is to design a simple domain-specific language in which traffic rules can be written and then processed using compiler techniques.

The system should be able to:

1. Read a TrafficRule program.
2. Identify the tokens present in the program.
3. Check the syntax using a grammar.
4. Build an Abstract Syntax Tree.
5. Check variables and values.
6. Maintain a symbol table.
7. Generate intermediate code.
8. Perform simple optimization.
9. Execute the rule using simulated data.
10. Show suitable error messages.

The project is kept small so that all the major stages can be implemented and demonstrated.

4 Motivation

Traffic rules are easy to represent using conditions. Values such as speed, signal colour and vehicle position can be treated as variables, while actions such as issuing a fine can be treated as statements.

This makes traffic rules a simple example for applying Compiler Design concepts.

Another reason for choosing this topic is that the output of each compiler stage can be shown during the demonstration. For example, the same rule can be displayed as source code, tokens, an AST and intermediate code.

The project also gives a simple way to demonstrate optimization without requiring a large programming language.

5 Objectives

The main objectives of the project are:

1. To design a small language for writing traffic rules.
2. To implement lexical analysis.
3. To implement syntax analysis using the defined grammar.
4. To construct an Abstract Syntax Tree.
5. To perform semantic checking.
6. To maintain a symbol table.
7. To generate intermediate representation.
8. To implement basic optimization.
9. To execute rules using simulated traffic values.
10. To identify and report different types of errors.

6 Scope of the Project

TrafficRule will mainly deal with simple traffic conditions and actions.

The language will support:

- Vehicle declaration.
- Variables such as speed, signal and position.
- Integer values.
- Traffic states.
- Comparison operators.
- Boolean operators.
- Conditional statements.
- Traffic-related actions.
- Intermediate representation.
- Basic optimization.
- Simulated execution.

The following features are not part of the project:

- Real traffic cameras.

- License plate recognition.
- GPS tracking.
- Real-time traffic data.
- YOLO or other vehicle detection models.
- Actual fine generation.
- Traffic signal hardware control.

The use of simulated data allows the project to concentrate mainly on compiler construction.

7 Background Study

A compiler normally performs a number of transformations before a program can be executed. TrafficRule will use the same basic idea.

7.1 Lexical Analysis

The lexer reads the source program and divides it into tokens.

For example:

```
speed = 75
```

can produce:

```
IDENTIFIER(speed)
ASSIGN
NUMBER(75)
```

Keywords such as `IF` and actions such as `ISSUE_FINE` will also be recognized as separate token types.

7.2 Syntax Analysis

The parser checks whether the tokens occur in a valid order.

For example:

```
IF speed > 60 {
    ISSUE_FINE
}
```

is a valid conditional rule, while a statement with a missing operand or closing brace should produce a syntax error.

7.3 Abstract Syntax Tree

The AST represents the structure of the program.

For the expression:

```
speed > 60
```

the structure can be viewed as:

```
      >
     / \
speed  60
```

The AST will be used by the later compiler stages.

7.4 Semantic Analysis

Semantic analysis checks whether a valid-looking statement also makes sense.

Some checks include:

- Checking whether a variable exists.
- Checking whether the assigned value is valid.
- Checking compatible operands.
- Checking supported actions.

7.5 Symbol Table

The symbol table stores information about identifiers used in the program.

A sample table is:

Name	Type	Value	Scope
speed	integer	75	global
signal	state	RED	global
position	state	CROSSING	global
car	vehicle	–	global

7.6 Intermediate Representation

After semantic checking, the AST can be converted into a simpler intermediate form.

For example:

```
t1 = speed > 60
IF_FALSE t1 GOTO L1
ISSUE_FINE
```

```
L1:
```

This representation makes the execution stage simpler.

7.7 Optimization

A small optimization stage will be included. It can be used to simplify constant conditions, remove unnecessary instructions and simplify Boolean expressions.

8 Compiler Design Concepts Involved

The major Compiler Design concepts used in the project are:

Concept	**Use in TrafficRule**
Lexical Analysis	Converts source code into tokens.
Syntax Analysis	Checks the grammar of the language.
AST	Represents the structure of rules.
Semantic Analysis	Checks variables, values and expressions.
Symbol Table	Stores identifier information.
Intermediate Code	Represents rules using simple instructions.
Optimization	Removes unnecessary operations.
Execution	Evaluates rules using simulated data.
Error Handling	Reports errors from different stages.

9 Proposed Methodology

The project will follow the following sequence:

1. **Source Code**

   The user writes a traffic rule using TrafficRule syntax.

2. **Lexer**

   The source code is converted into tokens.

3. **Parser**

   The tokens are checked against the grammar.

4. **AST**

   A tree representation of the program is created.

5. **Semantic Analysis**

   Variables, values and expressions are checked.

6. **Symbol Table**

   Information about identifiers is stored.

7. **Intermediate Code**

   The AST is converted into simple instructions.

8. **Optimization**

   Unnecessary instructions are simplified where possible.

9. **Execution**

   The final instructions are evaluated using simulated traffic data.

10. **Output**

    The system displays the result of the traffic rule.

10 System Design

The basic architecture of the system is:

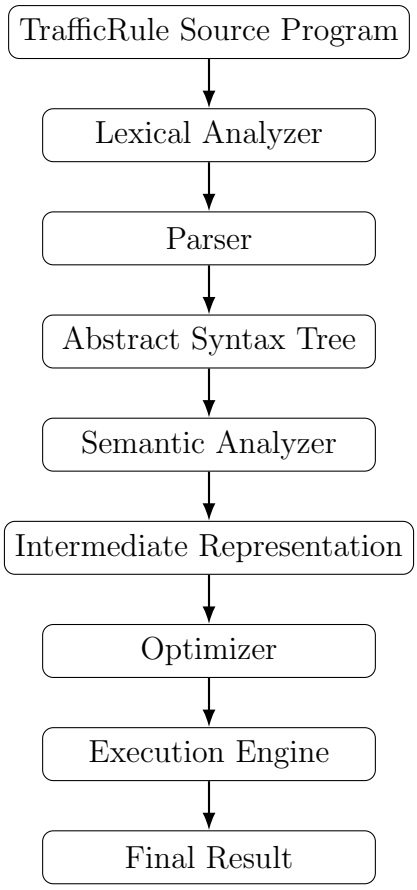


Figure 1: TrafficRule compiler architecture

The output from one stage becomes the input for the next stage. This allows the working of the compiler to be observed step by step.

11 Language Specification

TrafficRule will use a small set of keywords and operators.

11.1 Keywords

Keyword	Purpose
VEHICLE	Declares a vehicle
IF	Starts a condition
ELSE	Alternative branch
AND	Logical AND
OR	Logical OR
ISSUE_FINE	Issues a simulated fine
FLAG_VIOLATION	Marks a violation
ALLOW	Allows the condition

11.2 Operators

Operator	Meaning
=	Assignment
==	Equal to
!=	Not equal to
¿	Greater than
¡	Less than
¿=	Greater than or equal to
¡=	Less than or equal to

11.3 Traffic States

Some predefined values are:

```
RED
GREEN
YELLOW
CROSSING
ROAD
```

These values can be used in traffic conditions.

11.4 Initial Grammar

The initial grammar of the language is:

```
program       -> statement*
```

```
statement     -> declaration
               | assignment
               | if_stmt

declaration   -> VEHICLE identifier

assignment    -> identifier '=' value

if_stmt       -> IF condition block
                 (ELSE block)?

condition     -> expression
               | expression AND expression
               | expression OR expression

expression    -> value relational_op value

block         -> '{' statement* '}'

action        -> ISSUE_FINE
               | FLAG_VIOLATION
               | ALLOW

value         -> identifier
               | number
               | RED
               | GREEN
               | YELLOW
               | CROSSING
               | ROAD

relational_op -> '>'
               | '<'
               | '>='
               | '<='
               | '=='
               | '!='
```

The grammar can be extended slightly during implementation if required.

11.5 Sample Program

A TrafficRule program can be written as:

```
VEHICLE car

speed = 75
signal = RED
position = CROSSING

IF speed > 60 {
    ISSUE_FINE
}

IF signal == RED AND position == CROSSING {
    FLAG_VIOLATION
}
```

The first rule checks the speed of the vehicle. The second rule checks the signal and position together.

12 Module Description

The project will be divided into the following modules.

Module	Function
Lexer	Generates tokens.
Parser	Checks program syntax.
AST	Stores the structure of the program.
Semantic Analyzer	Performs semantic checks.
Symbol Table	Stores variables and their values.
IR Generator	Generates intermediate instructions.
Optimizer	Simplifies intermediate code.
Execution Engine	Evaluates the rules.
Error Handler	Displays errors.

12.1 Lexical Analyzer

The lexer will read the source program and identify different types of tokens.

Token	Example	Use
KEYWORD	IF	Language keyword
IDENTIFIER	speed	Variable name
NUMBER	75	Numeric value
OPERATOR	¿	Comparison
ASSIGN	=	Assignment
DELIMITER	{ }	Block boundary
STATE	RED	Traffic state
ACTION	ISSUE_FINE	Rule action

12.2 Parser

The parser receives the tokens and checks whether they match the grammar.

For example:

```
IF speed > 60 {
    ISSUE_FINE
}
```

will be recognized as an `IF` statement with a condition and an action.

12.3 Abstract Syntax Tree

The AST will represent a traffic rule in tree form.

For example:

```
       IF
        |
    Condition
        |
        >
      /   \
   speed   60
        |
      Action
        |
   ISSUE_FINE
```

12.4 Semantic Analyzer

The semantic analyzer will check whether the program is valid beyond its syntax.

It will check things such as:

- Undefined variables.
- Invalid values.
- Incorrect comparisons.
- Unsupported actions.

12.5 Symbol Table

The symbol table will store information about identifiers.

Name	Type	Value	Scope
speed	integer	75	global
signal	state	RED	global
position	state	CROSSING	global
car	vehicle	–	global

12.6 Intermediate Representation

The AST will be converted into intermediate instructions.

For example:

```
t1 = speed > 60
IF_FALSE t1 GOTO L1
ISSUE_FINE
L1:
```

For a Boolean condition:

```
t1 = signal == RED
t2 = position == CROSSING
t3 = t1 AND t2
IF_FALSE t3 GOTO L1
FLAG_VIOLATION
L1:
```

12.7 Optimization

The optimizer will perform simple transformations on the intermediate representation.

The initial optimization techniques will include:

- Constant condition evaluation.
- Boolean expression simplification.
- Removal of redundant instructions.
- Removal of unreachable branches.

12.8 Execution Engine

The execution engine will use simulated traffic values to evaluate the generated instructions.

For example:

```
speed = 75
```

with:

```
IF speed > 60 {
    ISSUE_FINE
}
```

will produce a simulated speed-violation result.

13 Error Handling

Errors will be handled according to the stage in which they occur.

Error	Example	Stage
Lexical Error	Invalid character	Lexer
Syntax Error	Missing brace	Parser
Semantic Error	Unknown variable	Semantic Analyzer
Type Error	Invalid comparison	Semantic Analyzer
Runtime Error	Invalid execution value	Execution Engine

For example:

```
IF speed > {
    ISSUE_FINE
}
```

should produce a syntax error because the comparison is incomplete.

14 Technology Stack

The initial technology choices are:

Technology	Purpose
Python / C++	Compiler implementation
Flex / Lex or Custom Lexer	Lexical analysis
Bison / Yacc or Custom Parser	Syntax analysis
Graphviz	AST visualization
CSV / JSON	Simulated traffic data
Git / GitHub	Version control
VS Code	Development

A custom lexer and parser can also be used if it makes the implementation simpler. The final choice will depend on the implementation approach.

15 Innovation

TrafficRule is different from a normal program because the user writes traffic rules using a language specifically designed for this purpose.

For example:

```
IF speed > 60 {
    ISSUE_FINE
}
```

Instead of directly executing this condition in a general-purpose language, TrafficRule processes it through compiler stages.

The project also makes the internal stages visible. Tokens, the AST, symbol table and intermediate code can be shown during the demonstration.

The use of simulated traffic data keeps the implementation manageable while allowing the language to be tested with different traffic situations.

16 Initial Prototype

The initial prototype will focus on the first parts of the compiler. A sample TrafficRule program will be given as input and the lexer and parser will process it.

The sample program is:

```
VEHICLE car

speed = 75
```

```
signal = GREEN
position = ROAD

IF speed > 60 {
    ISSUE_FINE
}
```

The prototype should demonstrate:

1. Token generation.
2. Syntax checking.
3. Basic AST structure.
4. Symbol-table information.
5. Simulated traffic input.

An example token output is:

```
VEHICLE     KEYWORD
car         IDENTIFIER
speed       IDENTIFIER
=           ASSIGN
75          NUMBER
signal      IDENTIFIER
=           ASSIGN
GREEN       STATE
position    IDENTIFIER
=           ASSIGN
ROAD        STATE
IF          KEYWORD
speed       IDENTIFIER
>           OPERATOR
60          NUMBER
{           DELIMITER
ISSUE_FINE  ACTION
}           DELIMITER
```

A possible simulated output is:

```
Traffic Violation Detected
Type: Speed Violation
Action: ISSUE_FINE
```

These are sample outputs for demonstrating the prototype.

17 Expected Outcome

The final system is expected to process a TrafficRule program from source code to execution.

The main expected features are:

- Token generation.
- Syntax checking.
- AST construction.
- Semantic analysis.
- Symbol-table management.
- Intermediate code generation.
- Basic optimization.
- Rule execution.
- Error reporting.

The Phase 1 prototype will be used as the starting point for implementing the remaining compiler stages and testing the language with different traffic-rule examples.